Supplementary Information for "Bounding the cost of federation: a pre-registered evaluation of synchronous and asynchronous federated object detection with disjoint class ownership in medical imaging"

Salvatore Capuozzo[1*], Stefano Marrone[1] and Carlo Sansone[1]

[1*]Department of Electrical Engineering and Information Technologies, University of Naples Federico II, Via Claudio 21, Naples, 80125, Italy.

*Corresponding author(s). E-mail(s): [Authors: e-mail];
Contributing authors: [Authors: e-mail]; [Authors: e-mail];

Abstract

This supplement describes the one negative case of the confirmatory study: the concurrent asynchronous scheme on ChestX-Det10. It gives the registered result on that use case and the verdict it implies, then covers the use case, every variant tried on it during development, the confirmatory result class by class, the analysis of its cause made after the confirmation, three fixes screened afterwards with a stopping rule fixed before their results, and the strategies in the literature. None of the fixes closed the gap. Sections, tables and equations of the main article are cited by their numbers there.

S1 Registered result and verdict

ChestX-Det10 is the third registered use case of the confirmatory study (Section 4.2.2 of the main article). Table S1 gives its confirmatory result, in the format of Table 4 of the main article, which reports kfm12, WBC and DENTEX.

The registered test requires the lower bound to exceed the margin on every use case it covers, and it tests the margins in a fixed sequence. It is applied to the three

Table S1 Confirmatory result on ChestX-Det10 (seeds 100–109, frozen code and configurations). *Fed:Cent*: federated / centralized mAP50 ×100, paired by seed, both after the same post-processing; mean, standard deviation and two-sided 95% t-interval. Loss: centralized minus federated mAP50, mean with its upper 95% bound. mAP50-95: the same ratio for mAP50-95, with its lower bound. Post-processing as in the main article: closing calibration for the sequential scheme, progressive calibration with refresh for the concurrent one.

Scheme	n	*Fed:Cent* (sd)	95% CI	Loss (upper)	mAP50-95 (low)
Sequential	10	97.8 (3.8)	95.0–100.5	+0.007 (+0.016)	96.2 (92.0)
Concurrent	10	83.2 (4.9)	79.7–86.7	+0.053 (+0.065)	83.5 (79.7)
Centralized mAP50 (calibrated)		0.312			

use cases registered first (kfm12, WBC, ChestX-Det10) and, as the headline claim (amendment 2), to all four, with DENTEX. Both tests give the same verdicts:

- *Sequential scheme: claim 90%.* Its lower bound here, 95.0%, exceeds both of its margins (85%, then 90%), as on kfm12 (96.7%), WBC (98.7%) and DENTEX (96.2%).
- *Concurrent scheme: no claim.* Its lower bound here, 79.7%, misses its first margin, 80%. This holds although its lower bounds on kfm12, WBC and DENTEX, 92.1%, 122.0% and 99.4%, exceed even 90%. On DENTEX alone, the held-out test, it passes both of its margins. ChestX-Det10 is thus the only use case on which a scheme failed.

The verdict is final. No seeds were added and no margin was changed.

This is a limitation of the concurrent scheme. Its guarantee depends on the data. We found no bug, and three fixes screened afterwards did not close the gap (Sections S5 and S6).

The rest of this supplement describes the case:

- the use case and what sets it apart (Section S2);
- every variant tried on it during development (Section S3);
- the confirmatory result, class by class and node by node (Section S4);
- the analysis of its cause, made after the confirmation (Section S5);
- three fixes screened after the confirmation (Section S6);
- strategies from the literature and their fit with rules R1–R3 (Section S7);
- what the case implies for deployment (Section S8).

Sections S5 and S6 are exploratory. They cannot change the registered verdict, and a repaired scheme would need a new pre-registered confirmation on new seeds.

S2 The use case

ChestX-Det10 [1] annotates ten findings on chest X-rays from NIH ChestX-ray14 [2]. It is prepared like the other use cases (Section 3.4 of the main article):

- 2,224 training and 555 evaluation images; 1,507 unique training images used;
- budget 56 epochs, from the plateau of the centralized pilot (55, rounded to whole rounds), that is, 28 rounds or updates per node.

Table S2 gives the class ownership.

Three properties set it apart from kfm12 and WBC:

- *Co-occurrence.* 53% of a node's images contain other nodes' findings, which that node sees unlabelled. The share is 22% on kfm12 and 0% on WBC. On DENTEX it is about 100%, yet the concurrent scheme keeps 99.7% there, so co-occurrence alone does not explain the loss. DENTEX lacks the other two properties: its 32 classes have 262–506 training boxes each, and it is close to the ceiling (centralized mAP50 0.959).
- *Ownership.* Two nodes own one large class each. The other two own four small classes each, with 134–749 training boxes per class.
- *Difficulty.* It has the lowest absolute accuracy (centralized mAP50 0.312 after calibration) and the longest budget.

Its longer budget also makes it the most expensive use case (confirmation runs, measured as in Section 5.2 of the main article):

- a federated run takes a median of 147–148 GPU minutes, against 119 for centralized, and 0.18 kWh;
- each node transfers 5.6 GB;
- the sequential scheme needs 3.15 times the wall-clock time of the concurrent one.

Table S2 Class ownership on ChestX-Det10 (training boxes in parentheses).

Node	Classes
A	Consolidation (2,024)
B	Effusion (1,666)
C	Nodule (749), Atelectasis (276), Calcification (267), Pneumothorax (169)
D	Fibrosis (603), Fracture (536), Emphysema (239), Mass (134)

S3 Development phase on ChestX-Det10 (exploratory)

The schemes of the main article were chosen on this use case, the one on which the parallel scheme failed. Table S3 gives the three schemes under the final protocol, in the format of Table 3 of the main article. Table S4 lists every variant run, with its result on this use case. The development phase used seeds 0–3 and is subject to the caveats of Section 4.2.1 of the main article.

Four findings shaped the schemes of the main article:

- *Merging models trained in parallel is what loses.* Every scheme that averages or merges updates computed from the same or a stale base model stayed at 72–90%, or lower where a variant broke training. Eighteen variants of the parallel and concurrent schemes did not close the gap: aggregation rules, update strength, losses, calibration and limits on drift. The only scheme that matched centralized applies each node's

Table S3 Development phase on ChestX-Det10 under the final protocol: *Fed:Cent* mean over seeds, with the lower end of its 95% CI in parentheses. Calibrated rows are compared with the centralized model given the same calibration.

Scheme	Post-processing	n	ChestX-Det10
Parallel synchronous	none	3	77.7 (74.2)
	closing calibration	3	79.9 (77.7)
Sequential synchronous	none	3	99.1 (88.7)
	closing calibration	3	97.6 (92.3)
Concurrent asynchronous	none	4	71.4 (52.8)
	progressive calibration	4	87.1 (82.5)
	progressive + refresh	4	87.8 (82.6)
Centralized mAP50 (seeds 0–3)			0.306

work on top of the previous node's result. Its final average combines only nearby versions.

- *Dilution is not the cause.* Stronger server steps and stale updates at full strength did not help, and the strongest settings broke training. A 4× server step gave 44%. Without staleness damping, BatchNorm running means drifted to $2.5 \cdot 10^5$ and mAP50 fell to 0.
- *Order matters.* In the early asynchronous runs, which trained one node at a time, rotating the node list moved the result of the same seed from 116% to 89%, 81% and 103%. The spread over node orders reached 3.6 times the spread over seeds. This is why the sequential scheme draws a new random order every cycle and averages its last four versions.
- *Calibration repairs part of the concurrent gap, not the parallel one.* Progressive calibration raised the concurrent scheme from 71% to 88%. The closing calibration raised the parallel scheme from 78% to 80%.

S4 Confirmatory result, class by class

Table S5 gives the per-class result of the confirmation for both schemes.

- *Sequential scheme.* Every class is at 89% or more, and every node at 94% or more.
- *Concurrent scheme.*
  - Five of the ten classes fall below 85%. The worst are Fracture (54%) and Pneumothorax (57%).
  - Seven of the ten paired differences exclude zero.
  - The loss falls on nodes C and D, which own four small classes each (80% and 69%). Nodes A and B, with one large class each, are at 101% and 91%.

The node explains the loss better than class size does. Retention correlates with training boxes at $\rho = +0.53$ ($p = 0.12$, not significant with ten classes). An interim analysis on seven seeds had given $\rho = +0.70$ ($p = 0.03$). Within node D, its largest class (Fibrosis, 603 boxes) is at 67%, and Emphysema (239 boxes) at 88%.

The scheme's overall 83.2% thus averages a node at 101% with one at 69%.

Table S4 Every development-phase variant, with its ChestX-Det10 *Fed:Cent* (%): mean over the seeds run, with the lower end of the 95% CI in parentheses when $n \geq 3$. "Cal.": with calibration, compared with the calibrated centralized model. Primary protocol: 20 epochs, constant learning rate. Final protocol: Section 4.1 of the main article.

Variant	Change	Seeds	ChestX-Det10
Primary protocol, parallel synchronous			
Primary method	FedAvg of shared layers, class rows from their owners, unowned classes masked	0–2	78 (74)
FedNova [3]	updates normalized by local steps	0–2	86 (66)
Exclusion loss	other classes trained as negatives at a node's own objects	0–2	87 (66)
Exclusion + distillation	adds the downloaded model's predictions for other classes	0–2	86 (61)
Fisher merging	diagonal Fisher-weighted merging	0–2	81 (72)
Pseudo-boxes	other classes' detections from the downloaded model used as labels	0–2	68 (50)
Fixed simplex head	class rows fixed to an equiangular frame	0–2	76 (63)
Calibration round	BatchNorm re-estimation and re-fit of each owner's rows	0–2	71 (68)
Primary protocol, one node at a time			
Primary method	each update applied on arrival, staleness-weighted, fixed order	0–2	116 (100)
CA2FL [4]	buffered updates with cached calibration	0–2	42 (12)
FedBuff [5]	buffered averaging	0–2	94 (82)
FedAsync [6]	update mixed in at 0.6, fixed order	0–2	110 (102)
Exclusion; + distillation	as above	0–2	103 (79); 101 (76)
Fixed simplex head	as above	0	81
Rotated node lists	the primary run with three other node orders	0	89, 81, 103
Random order + averaging	new order every cycle, last 4 versions averaged, cal.	0–3; 4–7	104 (CI-low); 109 (100)
Final protocol, parallel synchronous			
Parallel scheme	as in the main article	0–2	78 (74); cal. 80 (78)
Server step 2×, 4×	averaged update applied at twice or four times its size	0	78, 44; cal. 85, 81
Rotating models	4 models passed between nodes, averaged every cycle	0	75; cal. 78
Quarter-epoch rounds	4× more rounds of $\frac{1}{4}$ epoch, same compute	0	75
Exclusion loss	as above	0	78
Reshuffle control	data order reshuffled every round only	0	72
Final protocol, sequential and concurrent			
Sequential scheme	as in the main article	0–2	99 (89); cal. 98 (92)
Concurrent scheme	as in the main article; progressive cal., + refresh	0–2	77 (62); 88 (84); 89 (84)
Deadline stop	all nodes train until a common deadline	0	83; cal. 85
No staleness damping	stale updates at full strength	0	0; cal. 76
Mild damping	weight 0.63 at staleness 3	0	78; cal. 77
Quarter-epoch uploads	$\frac{1}{4}$-epoch updates, BatchNorm statistics interpolated	0	82
Exclusion loss	as above, BatchNorm statistics interpolated	0	76

The random-order scheme with averaging passed on seeds 0–3 against the plain centralized model and was withdrawn after it failed on fresh seeds 4–7 on WBC (lower bound 89.8% against 90%), not on ChestX-Det10 (Section 5.1.1 of the main article).

Table S5 Per-class result of the confirmation on ChestX-Det10 (seeds 100–109). Retention: federated AP50 / centralized AP50 of the class, %, means over seeds, both calibrated. Difference: concurrent minus centralized AP50, paired by seed, with its 95% CI. Mean retention per node (A / B / C / D): sequential 101 / 101 / 101 / 94, concurrent 101 / 91 / 80 / 69.

Node	Class	Boxes	Cent. AP50	Seq. (%)	Conc. (%)	Difference (95% CI)
A	Consolidation	2,024	0.509	101	101	+0.007 (−0.007, +0.022)
B	Effusion	1,666	0.463	101	91	−0.040 (−0.069, −0.010)
C	Nodule	749	0.263	104	90	−0.026 (−0.050, −0.001)
	Atelectasis	276	0.146	107	93	−0.010 (−0.029, +0.009)
	Calcification	267	0.430	94	79	−0.090 (−0.133, −0.047)
	Pneumothorax	169	0.119	101	57	−0.052 (−0.080, −0.023)
D	Fibrosis	603	0.190	94	67	−0.063 (−0.089, −0.038)
	Fracture	536	0.238	89	54	−0.110 (−0.140, −0.080)
	Emphysema	239	0.496	90	88	−0.059 (−0.129, +0.010)
	Mass	134	0.267	104	67	−0.088 (−0.152, −0.023)

S5 Why the concurrent scheme loses (exploratory)

After the confirmation we looked for the cause of the loss, for bugs, and for a fix. Except for the fix screens of Section S6, every number here comes from finished runs, read without retraining.

S5.1 Not the calibration, not the final model

Table S6 evaluates the confirmation runs at three stages, and with two choices of final model.

- *Calibration is not the cause.* The barrier-free progressive calibration ends where the closing calibration does (83.2% and 83.1%). Both re-fit every class row at full learning rate, so the deficit lies in the shared layers, not in the rows.
- *The final model is not the cause.* The registered final model averages the global versions right after each node's last update. The last global version alone is 7.3 points worse (sd 4.5, $n = 10$), with the loss again on nodes C and D. The averaging removes part of the damage.

Table S6 Concurrent scheme on the confirmation seeds of ChestX-Det10, by calibration stage and final model: *Fed:Cent* (%), mean over the ten seeds, with the per-node mean retention. The registered model is the average of the global versions right after each node's last update, evaluated after progressive calibration with refresh. These are means of per-seed ratios, so node values differ by 1–2 points from the ratios of means in Table S5.

Stage	Per-node average (registered)	Last global version
No calibration	77.4 (A 98, B 77, C 82, D 63)	67.3 (A 92, B 80, C 65, D 51)
Closing calibration	83.1 (A 101, B 92, C 84, D 70)	75.0 (A 97, B 91, C 70, D 60)
Progressive + refresh	83.2 (A 102, B 92, C 82, D 71)	75.8 (A 98, B 91, C 69, D 62)

S5.2 Code checks

We checked the concurrent path of the frozen code, which is what the confirmation ran. We found no bug:

- Staleness is computed as a version difference, and the virtual clock trains each update from the version current at its pull time.
- The delta rule, the variance clamp, each node's learning-rate schedule and the final per-node average behave as specified in Section 3.2 of the main article.

We found two design weaknesses. Neither explains the loss:

- *Staleness damping of class rows.* Owned class rows are damped by staleness like shared weights, although only their owner writes them. Calibration re-fits them before evaluation.
- *BatchNorm statistics accumulated as deltas.* After the first concurrent cycle this can drive many variances to zero (clamped), from the first layer on, which leaves the uncalibrated model unusable: mAP50 0.0 in one-cycle smoke runs. Training uses batch statistics, and calibration re-estimates the running ones, so calibrated results are unaffected.

S5.3 Hypotheses rejected

Table S7 lists the explanations we tested and rejected.

Table S7 Explanations tested and rejected. Development screens used seed 0, on which the concurrent scheme reaches 88% after progressive calibration.

Hypothesis	Test	Result
Nodes with little data finish early, and the rest of the federation erodes their classes (recency)	Class retention against the number of updates applied after the owner's last update, with node effects (100 class–seed pairs)	-0.05 points per update ($t = -0.3$). The raw correlation ($+0.58$) is the node's identity
Staleness damping (mean weight 0.47) leaves the model under-trained	Weaker or no damping	77–78% (mild); 0%, or 76% calibrated (none: BatchNorm unstable)
Smaller, less stale updates would help	Quarter-epoch uploads	82%
A common stop time would help	All nodes train until a deadline	85%, against 90% for the scheme on the same seed

S5.4 What class ownership protects

Rule R3 aggregates a class's rows only from its owner. That protects 0.04% of the model's parameters (Table S8). Every node writes all the others. A small, stable per-class loss therefore cannot come from ownership alone:

- *Sequential scheme.* It provides that stability implicitly. Every node revisits the shared layers every cycle, at full strength, and each update starts from the previous one. Its worst node is at 94%.

- *Concurrent scheme.* It has no such mechanism.

Table S8 Parameters of YOLO26-L with the classic head (24.8 M), by who writes them.

Part	Parameters	Share	Written by
Backbone and neck	23.4 M	94.3%	every node
Box branch	0.85 M	3.4%	every node
Class tower (classification branch before the rows)	0.55 M	2.2%	every node
Class rows	0.01 M	0.04%	the owner only

S5.5 Reading

The evidence points to the structure of the merge, on the data where it matters most:

- Each node's update moves the shared features towards its own findings. The images it trains on also contain other nodes' findings (53%), for which its masked loss gives no signal.
- Concurrent updates are computed from stale base models and added to the global model. They pull the shared features in different directions.
- The small classes have the weakest signal in the shared features, so their features erode first.
- The sequential scheme chains the updates, each starting from the result of the previous one, so no update computed from an outdated model is merged.

This reading is consistent with DENTEX, where the concurrent scheme keeps 99.7% although every image contains every node's classes. There all nodes train on the same images, no class is small, and the task is near its ceiling. We did not test which of these differences matters.

S6 Fixes screened after the confirmation (exploratory)

We screened three fixes. Each changes only the concurrent scheme and respects R1–R3 (Table S9).

Stopping rule, fixed before any result.

- Each fix first ran on development seed 3. On that seed the concurrent scheme shows the problem: 83.6%, worst node 70%. Seed 0 does not show it (91.5%, node D 102%), so it could not tell a fix from luck.
- *Pass:* at least 90% overall, and every node's classes at least 85% on average. The fix would then run seeds 1 and 2.
- *Fail:* nothing more is run for it.
- *After three seeds,* the fix would count as solving the case only if two conditions held:

Table S9 Fixes screened on the concurrent scheme.

Fix	Change	Cost
Final chain	The last 4 of each node's 28 cycles run after the concurrent part, one node at a time, in an order drawn from the seed (staleness 0)	same compute; wall-clock about +30%
Owned class towers	One copy of the class tower per node at each scale, trained only by its owner's classes and merged only from its owner (R3 extended from rows to towers). At initialization the output equals the standard model's	+7% parameters; the centralized reference would need the same towers
Conflict projection	Before an update is applied, its components that oppose the updates applied since its node pulled are removed, tensor by tensor, as in FedFV [7]. Agreeing components are kept. Rows, towers and BatchNorm statistics are left unchanged	server only, no data (R2 holds)

– its mean is at least 5 points above the concurrent scheme on the same seeds (86.5%), so at least 91.5%;
– its worst node averages at least 85%.

Only then would it be checked on kfm12 and WBC and confirmed anew.

One-cycle smoke runs on WBC first checked that each fix runs correctly in the training loop.

Result: all three fail

(Table S10).

- *Final chain and conflict projection* gain about 4 points overall. Each is a single run, and the concurrent scheme's results on this use case vary with sd 4.9 points between seeds. Node C stays at 71–77%.
- *Conflict projection was almost inactive.* It kept 99.9% of the update norm on average (minimum 99.2%), with no trend over the run. The nodes' updates rarely oppose each other tensor by tensor. Any conflict is finer-grained than whole tensors.
- *Owned class towers made node D worse* (60%). Isolating the 2.2% of parameters in the class towers removes what node D's classes gained from the other nodes' data. It also leaves the 94% in backbone and neck as exposed as before. The deficit lies in the shared backbone and neck.

None of the three solves the case. Per the stopping rule, no further seeds, use cases or confirmation were run.

S7 Strategies from the literature

Table S11 groups the strategies we considered, with their fit with R1–R3 and what was tried here.

Class-level fixes act on the class rows, but the deficit lies in the shared features (Section S5), so they cannot close it on their own. Rare-class resampling would also have to be applied to the centralized reference. Two directions remain untested under

Table S10 Fix screens on ChestX-Det10, development seed 3, progressive calibration with refresh: *Fed:Cent* (%) and mean retention of each node's classes.

Variant	*Fed:Cent*	A	B	C	D	Rule
Concurrent scheme (reference)	83.6	102	95	78	70	–
Final chain	87.8	99	99	77	79	fail
Owned class towers	81.5	97	102	80	60	fail
Conflict projection	87.6	93	105	71	84	fail

Table S11 Strategies against the loss of rare classes under disjoint ownership.

Family	Idea	Literature	Fit	Tried here
Feature ownership (parameter isolation)	One class tower or encoder per owner, aggregated only from its owner	FedWeIT [8], Fed-MENU [9]	full	owned towers: worse
Server-side conflict correction	Remove from a stale update the components opposing recent updates, or compensate its delay	FedFV [7], DC-ASGD [10]	full	projection: inactive
Knowledge preservation in local training	Distil the pulled model's outputs for the classes a node does not own	FedNTD [11], ConDistFL [12]	full; about +30% compute	partly: distillation (86%), pseudo-boxes (68%), primary protocol
Importance-weighted protection	Penalize moving parameters important to other nodes, from shared Fisher diagonals	FedCurv [13]	full; communication ×3	server-side Fisher merging: 81%
Class-level fixes	Rare-class resampling, restricted softmax, logit calibration, classifier re-training	RFS [14], FedRS [15], FedLC [16], CReFF [17]	full	masking (≈ FedRS) and calibration (≈ CReFF) are part of the system
Replicated ownership, coded redundancy	Two owners per class; parity data at the server	coded computing [18]	breaks R3 or R2	not applicable

the final protocol: feature-level distillation in local training, and conflict correction finer than whole tensors.

S8 Implications

- *For the claims of the main article.* The concurrent scheme has no registered guarantee. Its lower bounds on kfm12, WBC and DENTEX show that it can be non-inferior. Whether it is depends on the data, and we know of no way to tell in advance other than measuring it.
- *Warning signs from this case:*
  - many objects of other nodes' classes in a node's images, which it sees unlabelled, together with the next sign (on DENTEX this share is about 100% and harmless on its own);
  - small classes concentrated on a few nodes;
  - a wide spread of per-node retention in a pilot run.

- *For deployment.* On such data, use the sequential scheme. It kept every node at 94% or more here. Alternatively, compare the per-node retention of a concurrent and a sequential pilot run before choosing concurrency.
- *For evaluation.* The concurrent scheme's mean (83%) hid a node at 69%. Per-node and worst-class retention should be part of the acceptance criteria, as argued in Section 5.2 of the main article.

S9 Reproducibility

Paths are relative to the code release.

- *Data.* ChestX-Det10 is converted to YOLO format by `tools/convert_chestx_det10.py`, which is lossless in images and boxes. It is split and fingerprinted like the other use cases (Appendix A of the main article).
- *Analysis.* The analysis of this case is `analysis/CHESTX_CONCURRENT.md`. The development search, with every variant, is `analysis/SEARCH_REPORT.md`.
- *Tables.*
  - Table S5: `tools/fairness_analysis.py --confirm`.
  - Tables S6 and S10: `tools/cx_diagnosis.py`.
- *Stopping rule.* `tools/cx_gate.py`; its decisions are in `runs/variants/_cx_gate.json`.
- *Fixes.* The options `federation.async_final_chain`, `owned_towers` and `async_conflict` are in `fedyolo/config.py`.
  - Configurations: `configs/variants/cx_chain4`, `cx_towers`, `cx_proj`, and `diag_cx_last` for the last-version evaluation.
  - These options were added after the confirmation and are not part of the frozen confirmatory copy.

References

[1] Liu, J., Lian, J., Yu, Y.: ChestX-Det10: Chest X-ray dataset on detection of thoracic abnormalities. arXiv preprint arXiv:2006.10550 (2020) https://doi.org/10.48550/arXiv.2006.10550

[2] Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M., Summers, R.M.: ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3462–3471 (2017). https://doi.org/10.1109/CVPR.2017.369

[3] Wang, J., Liu, Q., Liang, H., Joshi, G., Poor, H.V.: Tackling the objective inconsistency problem in heterogeneous federated optimization. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33 (2020). https://doi.org/10.48550/arXiv.2007.07481

[4] Wang, Y., Cao, Y., Wu, J., Chen, R., Chen, J.: Tackling the data heterogeneity in asynchronous federated learning with cached update calibration. In: International Conference on Learning Representations (ICLR) (2024). No DOI or arXiv record. https://iclr.cc/virtual/2024/poster/19456

[5] Nguyen, J., Malik, K., Zhan, H., Yousefpour, A., Rabbat, M., Malek, M., Huba, D.: Federated learning with buffered asynchronous aggregation. In: Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 151, pp. 3581–3607 (2022). https://doi.org/10.48550/arXiv.2106.06639

[6] Xie, C., Koyejo, S., Gupta, I.: Asynchronous federated optimization. arXiv preprint arXiv:1903.03934 (2019) https://doi.org/10.48550/arXiv.1903.03934

[7] Wang, Z., Fan, X., Qi, J., Wen, C., Wang, C., Yu, R.: Federated learning with fair averaging. In: Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1615–1623 (2021). https://doi.org/10.24963/ijcai.2021/223

[8] Yoon, J., Jeong, W., Lee, G., Yang, E., Hwang, S.J.: Federated continual learning with weighted inter-client transfer. In: Proceedings of the 38th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 139 (2021). https://doi.org/10.48550/arXiv.2003.03196

[9] Xu, X., Deng, H.H., Gateno, J., Yan, P.: Federated multi-organ segmentation with inconsistent labels. IEEE Transactions on Medical Imaging **42**(10), 2948–2960 (2023) https://doi.org/10.1109/TMI.2023.3270140

[10] Zheng, S., Meng, Q., Wang, T., Chen, W., Yu, N., Ma, Z.-M., Liu, T.-Y.: Asynchronous stochastic gradient descent with delay compensation. In: Proceedings of the 34th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 70 (2017). https://doi.org/10.48550/arXiv.1609.08326

[11] Lee, G., Jeong, M., Shin, Y., Bae, S., Yun, S.-Y.: Preservation of the global knowledge by not-true distillation in federated learning. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 38461–38474 (2022). https://doi.org/10.52202/068431-2787

[12] Wang, P., Shen, C., Wang, W., Oda, M., Fuh, C.-S., Mori, K., Roth, H.R.: ConDistFL: Conditional distillation for federated learning from partially annotated data. In: Medical Image Computing and Computer Assisted Intervention – MICCAI 2023 Workshops. Lecture Notes in Computer Science, pp. 311–321. Springer, ??? (2023). https://doi.org/10.1007/978-3-031-47401-9_30

[13] Shoham, N., Avidor, T., Keren, A., Israel, N., Benditkis, D., Mor-Yosef, L., Zeitak, I.: Overcoming forgetting in federated learning on non-IID data. arXiv preprint

arXiv:1910.07796 (2019) https://doi.org/10.48550/arXiv.1910.07796

[14] Gupta, A., Dollár, P., Girshick, R.: LVIS: A dataset for large vocabulary instance segmentation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5351–5359 (2019). https://doi.org/10.1109/CVPR.2019.00550

[15] Li, X.-C., Zhan, D.-C.: FedRS: Federated learning with restricted softmax for label distribution non-IID data. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 995–1005 (2021). https://doi.org/10.1145/3447548.3467254

[16] Zhang, J., Li, Z., Li, B., Xu, J., Wu, S., Ding, S., Wu, C.: Federated learning with label distribution skew via logits calibration. In: Proceedings of the 39th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 162 (2022). https://doi.org/10.48550/arXiv.2209.00189

[17] Shang, X., Lu, Y., Huang, G., Wang, H.: Federated learning on heterogeneous and long-tailed data via classifier re-training with federated features. In: Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI), pp. 2218–2224 (2022). https://doi.org/10.24963/ijcai.2022/308

[18] Prakash, S., Dhakal, S., Akdeniz, M.R., Yona, Y., Talwar, S., Avestimehr, S., Himayat, N.: Coded computing for low-latency federated learning over wireless edge networks. IEEE Journal on Selected Areas in Communications **39**(1), 233–250 (2021) https://doi.org/10.1109/JSAC.2020.3036961